CP7_The Insight Collective

Mekdelawit Dejene, Hanna Weinstein, Ayman Ibenbrahim, Lincoln Chank

2026-04-09

Executive Summary

This document outlines the data preparation, text preprocessing, feature engineering, labeling, and data partitioning steps performed to generate a structured list of professional skills from a profile's current and previous job descriptions on the Kogod Career Development LinkedIn profile dataset.

```
# install required packages
# install.packages(c("readr", "readxl", "dplyr", "tidyr", "stringr", "tidytext",
#                    "SnowballC", "stopwords", "tm", "ggplot2", "forcats",
#                    "scales", "wordcloud", "ggraph", "igraph", "tidyverse"))
```

```
# Libraries used for this project

library(readr)       # used to read csv files into R so we can start working with the data
library(readxl)      # used to read Excel files if we need any xlsx inputs
library(dplyr)       # used to filter, select, mutate, count, and summarize the data
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)       # used to reshape data and split/combine columns like separate(), unite(), and piv
library(stringr)     # used to clean text like making everything lowercase and removing extra character
library(tidytext)    # used for text mining steps like tokenizing words, removing stopwords, and creati
library(SnowballC)   # used for stemming so similar words get reduced to the same root form
library(stopwords)   # used to pull common stopword lists like 'the', 'and', and 'of'
library(tm)          # text mining package that supports document-style text processing
```

```
## Loading required package: NLP

##
## Attaching package: 'tm'
```

```
## The following object is masked from 'package:stopwords':
##
##     stopwords
```

```r
library(ggplot2)     # used to make charts and visual checks
```

```
##
## Attaching package: 'ggplot2'

## The following object is masked from 'package:NLP':
##
##     annotate
```

```r
library(forcats)     # used to reorder categories in charts so plots are easier to read
library(scales)      # used to format chart axes and percentages so outputs are cleaner
```

```
##
## Attaching package: 'scales'

## The following object is masked from 'package:readr':
##
##     col_factor
```

```r
library(wordcloud)   # used if we want to create a word cloud for quick text patterns
```

```
## Loading required package: RColorBrewer
```

```r
library(tidyverse)   # bundle of core packages for wrangling and exploring the data
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v lubridate 1.9.4     v tibble    3.3.0
## v purrr     1.1.0

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x ggplot2::annotate()  masks NLP::annotate()
## x scales::col_factor() masks readr::col_factor()
## x purrr::discard()     masks scales::discard()
## x dplyr::filter()      masks stats::filter()
## x dplyr::lag()         masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(ggraph)      # used for network-style visualizations if needed later
library(igraph)      # used to build and work with graph/network objects if needed later
```

```
##
## Attaching package: 'igraph'
##
## The following objects are masked from 'package:lubridate':
##
```

```
##     %--%, union
##
## The following objects are masked from 'package:purrr':
##
##     compose, simplify
##
## The following object is masked from 'package:tibble':
##
##     as_data_frame
##
## The following object is masked from 'package:tidyr':
##
##     crossing
##
## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union
##
## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
##
## The following object is masked from 'package:base':
##
##     union
```

Part 1: Loading and Checking Data

Data Cleaning Pipeline

GitHub Repository: https://github.com/tangiraiC/ITEC621-practicum

```r
# To reproduce the cleaned data, run quarto::quarto_render("datacleaning.qmd") before running this scri

# Load the cleaned datasets that we will use throughout the rest of the assignment
exp_clean <- read.csv("data/processed/cleaned_500_experiences.csv",
                      stringsAsFactors = FALSE)
currexp_clean <- read.csv("data/processed/cleaned_500_current_experiences.csv",
                          stringsAsFactors = FALSE)
skills_clean <- read.csv("data/processed/cleaned_500_skills.csv",
                         stringsAsFactors = FALSE)
exp_5k_clean <- read.csv("data/processed/cleaned_5k_experiences.csv",
                         stringsAsFactors = FALSE)
skills_5k_clean <- read.csv("data/processed/cleaned_5k_skills.csv",
                            stringsAsFactors = FALSE)
```

Inspecting Dataset Dimensions

```r
# Resource used to help: https://www.w3schools.com/r/r_data_frames.asp
```

```r
# Check how many rows and columns each dataset has so we know the size of each file before modeling
file_summary <- cbind(
  File = c("500 experiences", "500 current experiences","500 skills", "5k experiences", "5k skills"),
  Rows = c(nrow(exp_clean), nrow(currexp_clean), nrow(skills_clean), nrow(exp_5k_clean), nrow(skills_5k
  Columns = c(ncol(exp_clean), ncol(currexp_clean), ncol(skills_clean), ncol(exp_5k_clean), ncol(skills

print(file_summary)
```

```
##      File                      Rows    Columns
## [1,] "500 experiences"         "452"   "58"
## [2,] "500 current experiences" "452"   "6"
## [3,] "500 skills"              "6364"  "2"
## [4,] "5k experiences"          "5865"  "13"
## [5,] "5k skills"               "77128" "2"
```

```r
# Output meaning: this table shows the size of each dataset and helps us confirm the files loaded corre
```

Inspecting Schema and Structure

```r
# Check the column names so we can see what variables are available in each dataset
names(exp_clean)
```

```
##  [1] "linkedinUrl"     "jobDescription"  "jobTitle"        "companyName"
##  [5] "jobStartedOn_1"  "jobStartedOn_2"  "jobStartedOn_3"  "jobStartedOn_4"
##  [9] "jobStartedOn_5"  "jobStartedOn_6"  "jobStartedOn_7"  "jobStartedOn_8"
## [13] "jobStartedOn_9"  "jobStartedOn_10" "jobStartedOn_11" "jobStartedOn_12"
## [17] "jobStartedOn_13" "jobStartedOn_14" "jobStartedOn_15" "jobStartedOn_16"
## [21] "jobStartedOn_17" "jobStartedOn_18" "jobStartedOn_19" "jobStartedOn_20"
## [25] "jobStartedOn_21" "jobStartedOn_22" "jobStartedOn_23" "jobStartedOn_24"
## [29] "jobStartedOn_25" "jobStartedOn_26" "jobStartedOn_27" "jobEndedOn_1"
## [33] "jobEndedOn_2"    "jobEndedOn_3"    "jobEndedOn_4"    "jobEndedOn_5"
## [37] "jobEndedOn_6"    "jobEndedOn_7"    "jobEndedOn_8"    "jobEndedOn_9"
## [41] "jobEndedOn_10"   "jobEndedOn_11"   "jobEndedOn_12"   "jobEndedOn_13"
## [45] "jobEndedOn_14"   "jobEndedOn_15"   "jobEndedOn_16"   "jobEndedOn_17"
## [49] "jobEndedOn_18"   "jobEndedOn_19"   "jobEndedOn_20"   "jobEndedOn_21"
## [53] "jobEndedOn_22"   "jobEndedOn_23"   "jobEndedOn_24"   "jobEndedOn_25"
## [57] "jobEndedOn_26"   "jobEndedOn_27"
```

```r
names(exp_5k_clean)
```

```
##  [1] "linkedin_url"             "headline"
##  [3] "field_of_study"           "current_job_title"
##  [5] "current_job_description"  "current_company_name"
##  [7] "current_job_industry"     "previous_job_title"
##  [9] "previous_job_description" "previous_company_name"
## [11] "previous_job_Industry"    "Skills"
## [13] "total_experience_count"
```

```r
names(skills_5k_clean)
```

```
## [1] "linkedin_url" "Skills"
```

```r
# Check the structure so we can verify data types, variable formats, and a sample of values
str(exp_clean)
```

```
## 'data.frame':    452 obs. of  58 variables:
##  $ linkedinUrl    : chr  "https://www.linkedin.com/in/dale-smith-18103b26/" "https://www.linkedin.co
##  $ jobDescription : chr  "Asset Manager for the Calvert Mine, Twin Oaks Power Station, and Big Rooter
##  $ jobTitle       : chr  "Vice President, Asset Management Plant Manager Plant Manager of Patua Geotl
##  $ companyName    : chr  "Panamint Capital Open Mountain Energy Cyrq Energy Cyrq Energy, Inc. Alterra
##  $ jobStartedOn_1 : chr  "12-2023" NA "7-2024" "7-2022" ...
##  $ jobStartedOn_2 : chr  "2-2023" "5-1998" "10-2022" "1-2016" ...
##  $ jobStartedOn_3 : chr  "7-2021" "7-1997" "8-2021" "7-2014" ...
##  $ jobStartedOn_4 : chr  "1-2015" "9-1995" "9-2016" "8-2012" ...
##  $ jobStartedOn_5 : chr  "8-2010" "3-1991" "8-2015" NA ...
##  $ jobStartedOn_6 : chr  "10-2009" NA "6-2014" NA ...
##  $ jobStartedOn_7 : chr  "1-2009" NA NA NA ...
##  $ jobStartedOn_8 : chr  "10-2006" NA NA NA ...
##  $ jobStartedOn_9 : chr  "1-2006" NA NA NA ...
##  $ jobStartedOn_10: chr  "4-2004" NA NA NA ...
##  $ jobStartedOn_11: chr  "8-2002" NA NA NA ...
##  $ jobStartedOn_12: chr  "6-2002" NA NA NA ...
##  $ jobStartedOn_13: chr  "6-2001" NA NA NA ...
##  $ jobStartedOn_14: chr  "1-2001" NA NA NA ...
##  $ jobStartedOn_15: chr  "6-1996" NA NA NA ...
##  $ jobStartedOn_16: chr  NA NA NA NA ...
##  $ jobStartedOn_17: chr  NA NA NA NA ...
##  $ jobStartedOn_18: chr  NA NA NA NA ...
##  $ jobStartedOn_19: chr  NA NA NA NA ...
##  $ jobStartedOn_20: chr  NA NA NA NA ...
##  $ jobStartedOn_21: chr  NA NA NA NA ...
##  $ jobStartedOn_22: chr  NA NA NA NA ...
##  $ jobStartedOn_23: chr  NA NA NA NA ...
##  $ jobStartedOn_24: int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobStartedOn_25: int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobStartedOn_26: int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobStartedOn_27: chr  NA NA NA NA ...
##  $ jobEndedOn_1   : chr  NA NA NA "12-2023" ...
##  $ jobEndedOn_2   : chr  "12-2023" NA "5-2024" "7-2022" ...
##  $ jobEndedOn_3   : chr  "2-2023" "5-1998" "10-2022" "12-2015" ...
##  $ jobEndedOn_4   : chr  "7-2021" "7-1997" "8-2021" "2-2014" ...
##  $ jobEndedOn_5   : chr  "1-2015" "9-1995" "8-2016" NA ...
##  $ jobEndedOn_6   : chr  "8-2010" NA "8-2015" NA ...
##  $ jobEndedOn_7   : chr  "10-2009" NA NA NA ...
##  $ jobEndedOn_8   : chr  "12-2008" NA NA NA ...
##  $ jobEndedOn_9   : chr  "10-2006" NA NA NA ...
##  $ jobEndedOn_10  : chr  "1-2006" NA NA NA ...
##  $ jobEndedOn_11  : chr  "4-2004" NA NA NA ...
##  $ jobEndedOn_12  : chr  "8-2002" NA NA NA ...
##  $ jobEndedOn_13  : chr  "6-2002" NA NA NA ...
##  $ jobEndedOn_14  : chr  "5-2001" NA NA NA ...
```

```
##  $ jobEndedOn_15  : chr  "12-2000" NA NA NA ...
##  $ jobEndedOn_16  : chr  NA NA NA NA ...
##  $ jobEndedOn_17  : chr  NA NA NA NA ...
##  $ jobEndedOn_18  : chr  NA NA NA NA ...
##  $ jobEndedOn_19  : chr  NA NA NA NA ...
##  $ jobEndedOn_20  : chr  NA NA NA NA ...
##  $ jobEndedOn_21  : chr  NA NA NA NA ...
##  $ jobEndedOn_22  : chr  NA NA NA NA ...
##  $ jobEndedOn_23  : chr  NA NA NA NA ...
##  $ jobEndedOn_24  : int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobEndedOn_25  : int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobEndedOn_26  : int  NA NA NA NA NA NA NA NA NA NA ...
##  $ jobEndedOn_27  : chr  NA NA NA NA ...
```

```r
str(exp_5k_clean)
```

```
## 'data.frame':    5865 obs. of  13 variables:
##  $ linkedin_url            : chr  "https://www.linkedin.com/in/dalejankowski/" "https://www.linkedin
##  $ headline                : chr  "Life Transitions and Estate Sale Services" "Engineer" "Program Ma
##  $ field_of_study          : chr  "BA, Psychology" "Master of Science (SM), System Design and Manage
##  $ current_job_title       : chr  "Legacy Life Transitions Manager" "Senior Backend Engineer" "Retir
##  $ current_job_description : chr  NA NA NA "•\tEvaluated sites, both remote and in-person, for proje
##  $ current_company_name    : chr  "Legacy Life Transitions" "RTK Tickets" "Federal Government" "Dist
##  $ current_job_industry    : chr  "Wholesale Building Materials" "Retail" "Government Administration
##  $ previous_job_title      : chr  "Realtor" "Senior Product Engineer" NA "Lecturer" ...
##  $ previous_job_description: chr  "•       Seniors Real Estate Specialist designation\n•\tSourced an
##  $ previous_company_name   : chr  "Crye Leike Realtors" "Pawling Engineered Products, Inc." NA "Cali
##  $ previous_job_Industry   : chr  NA NA NA NA ...
##  $ Skills                  : chr  "Communication, Project Management, Teamwork, Customer Satisfactio
##  $ total_experience_count  : int  8 4 3 4 6 6 15 5 4 2 ...
```

```r
str(skills_5k_clean)
```

```
## 'data.frame':    77128 obs. of  2 variables:
##  $ linkedin_url: chr  "https://www.linkedin.com/in/dalejankowski/" "https://www.linkedin.com/in/dale
##  $ Skills      : chr  "Communication" "Project Management" "Teamwork" "Customer Satisfaction" ...
```

Part 2: Data Quality and Validation

```r
# Running basic data quality checks to see missing values, duplicates, and whether key fields are usabl

# Counts missing values per dataset and per column, then checks for duplicate or missing LinkedIn URLs
sum(is.na(exp_clean))
```

```
## [1] 18965
```

```r
sum(is.na(exp_5k_clean))
```

```
## [1] 18172
```

```r
sum(is.na(skills_5k_clean))
```

```
## [1] 0
```

```r
# Check which columns contain missing values so we know where the gaps are coming from
colSums(is.na(exp_clean))
```

```
##     linkedinUrl  jobDescription        jobTitle     companyName  jobStartedOn_1
##               0               0               0               0              19
##  jobStartedOn_2  jobStartedOn_3  jobStartedOn_4  jobStartedOn_5  jobStartedOn_6
##              28              65              95             143             195
##  jobStartedOn_7  jobStartedOn_8  jobStartedOn_9 jobStartedOn_10 jobStartedOn_11
##             250             285             329             364             385
## jobStartedOn_12 jobStartedOn_13 jobStartedOn_14 jobStartedOn_15 jobStartedOn_16
##             402             414             424             431             440
## jobStartedOn_17 jobStartedOn_18 jobStartedOn_19 jobStartedOn_20 jobStartedOn_21
##             444             448             450             450             450
## jobStartedOn_22 jobStartedOn_23 jobStartedOn_24 jobStartedOn_25 jobStartedOn_26
##             450             451             451             451             451
## jobStartedOn_27    jobEndedOn_1    jobEndedOn_2    jobEndedOn_3    jobEndedOn_4
##             451             395             110              97             110
##    jobEndedOn_5    jobEndedOn_6    jobEndedOn_7    jobEndedOn_8    jobEndedOn_9
##             151             200             253             286             331
##   jobEndedOn_10   jobEndedOn_11   jobEndedOn_12   jobEndedOn_13   jobEndedOn_14
##             367             387             403             415             424
##   jobEndedOn_15   jobEndedOn_16   jobEndedOn_17   jobEndedOn_18   jobEndedOn_19
##             432             441             444             448             450
##   jobEndedOn_20   jobEndedOn_21   jobEndedOn_22   jobEndedOn_23   jobEndedOn_24
##             450             450             450             451             451
##   jobEndedOn_25   jobEndedOn_26   jobEndedOn_27
##             451             451             451
```

```r
colSums(is.na(exp_5k_clean))
```

```
##             linkedin_url                 headline           field_of_study
##                        0                      121                      849
##        current_job_title  current_job_description     current_company_name
##                      461                     3815                      505
##     current_job_industry       previous_job_title previous_job_description
##                     1699                     1621                     3505
##    previous_company_name    previous_job_Industry                   Skills
##                     1630                     2819                     1028
##   total_experience_count
##                      119
```

```r
# Output meaning: most missing values are in timing or background fields like job dates, industry, and
# Our main fields for this project, like LinkedIn URL and job description, are clean.
```

```r
# Check for duplicate LinkedIn URLs because each profile should ideally appear once
sum(duplicated(exp_clean$linkedinUrl))
```

```
## [1] 0
```

```r
sum(duplicated(exp_5k_clean$linkedinUrl))
```

```
## [1] 0
```

```r
# Output meaning: no duplicate LinkedIn URLs showed up here, so profile IDs look clean.
```

```r
# Check for missing LinkedIn URLs because we need them as profile identifiers
sum(is.na(exp_clean$linkedinUrl))
```

```
## [1] 0
```

```r
sum(is.na(exp_5k_clean$linkedin_url))
```

```
## [1] 0
```

```r
# Output meaning: there are no missing profile links in these key files.
```

```r
# Check how many profiles actually have a usable job description in both datasets
sum(!is.na(exp_clean$jobDescription) & exp_clean$jobDescription !="")
```

```
## [1] 316
```

```r
sum(!is.na(exp_5k_clean$current_job_description) & exp_5k_clean$current_job_description !="")
```

```
## [1] 2050
```

```r
# Turn that count into a percentage so it is easier to compare coverage across datasets
sum(!is.na(exp_clean$jobDescription) & exp_clean$jobDescription != "") /
nrow(exp_clean) * 100
```

```
## [1] 69.9115
```

```r
sum(!is.na(exp_5k_clean$current_job_description) & exp_5k_clean$current_job_description !="")/ nrow(exp
```

```
## [1] 34.95311
```

```r
# Output meaning: Counts non-empty job descriptions in 500 vs 5k datasets, converts to percentages, and

# The 5k dataset will be used to fill in gaps as needed.
```

Part 3: Text Preprocessing

```r
# Start building the text analysis dataset using the professor's text analytics workflow as a guide
# Keep only profiles with a real job description and make a clean text table to work from
job_text <- exp_clean |>
  filter(!is.na(jobDescription) & jobDescription != "") |>
  mutate(line_id = row_number()) |>
  select(linkedinUrl, jobDescription, line_id)

cat("Profiles with job descriptions:", nrow(job_text), "\n")
```

```
## Profiles with job descriptions: 316
```

```r
# Output meaning: this tells us how many profiles actually moved into the text preprocessing stage.
cat("\nBefore cleaning:\n")
```

```
##
## Before cleaning:
```

```r
cat(job_text$jobDescription[1], "\n")
```

```
## Asset Manager for the Calvert Mine, Twin Oaks Power Station, and Big Rooter Solar.
```

```r
# Step 1: normalize the text so everything is in a consistent format before tokenizing
  # This part lowercases the text, removes bullets and line breaks, drops punctuation/numbers, and trim
job_text <- job_text |>
  mutate(text_lower = str_to_lower(jobDescription),
    text_clean = str_remove_all(text_lower, "•|\\\\t|\\\\n|\\\\r"),
    text_clean = str_remove_all(text_clean, "[^a-zA-Z\\s]"),
    text_clean = str_squish(text_clean))

cat("After normalization:\n")
```

```
## After normalization:
```

```r
cat(job_text$text_clean[1], "\n")
```

```
## asset manager for the calvert mine twin oaks power station and big rooter solar
```

```r
# Quick quality check to compare the original job description with the cleaned version
job_text |>
  filter(str_detect(jobDescription, "[^a-zA-Z\\s]")) |>
  head(6) |>
  select(line_id, jobDescription, text_clean)
```

```
##   line_id
## 1       1
## 2       2
## 3       3
## 4       4
## 5       5
```

```
## 6       6
##
## 1
## 2 Principal Global Investors is a diversified global asset management firm and a member of the Princi
## 3
## 4
## 5
## 6
##
## 1
## 2 principal global investors is a diversified global asset management firm and a member of the princi
## 3
## 4
## 5
## 6
```

```r
# Output meaning: the cleaned example is now lowercase, simpler, and easier to tokenize.
```

```r
# Step 2: split each cleaned description into individual words (tokens)
job_words <- job_text |>
  select(linkedinUrl, line_id, text_clean) |>
  unnest_tokens(output = word, input = text_clean, token = "words")

# Quick check so we can see how many total tokens we created and what they look like
cat("Total tokens:", nrow(job_words), "\n")
```

```
## Total tokens: 92499
```

```r
cat("Average tokens per profile:", round(nrow(job_words)/nrow(job_text), 1), "\n")
```

```
## Average tokens per profile: 292.7
```

```r
job_words |> head(15)
```

```
##                                        linkedinUrl line_id      word
## 1  https://www.linkedin.com/in/dale-smith-18103b26/       1     asset
## 2  https://www.linkedin.com/in/dale-smith-18103b26/       1   manager
## 3  https://www.linkedin.com/in/dale-smith-18103b26/       1       for
## 4  https://www.linkedin.com/in/dale-smith-18103b26/       1       the
## 5  https://www.linkedin.com/in/dale-smith-18103b26/       1   calvert
## 6  https://www.linkedin.com/in/dale-smith-18103b26/       1      mine
## 7  https://www.linkedin.com/in/dale-smith-18103b26/       1      twin
## 8  https://www.linkedin.com/in/dale-smith-18103b26/       1      oaks
## 9  https://www.linkedin.com/in/dale-smith-18103b26/       1     power
## 10 https://www.linkedin.com/in/dale-smith-18103b26/       1   station
## 11 https://www.linkedin.com/in/dale-smith-18103b26/       1       and
## 12 https://www.linkedin.com/in/dale-smith-18103b26/       1       big
## 13 https://www.linkedin.com/in/dale-smith-18103b26/       1    rooter
## 14 https://www.linkedin.com/in/dale-smith-18103b26/       1     solar
## 15            https://www.linkedin.com/in/daleward/       2 principal
```

```r
# Output meaning: Splits cleaned descriptions into word tokens, counts total tokens and average tokens p
```

```r
# Step 3: remove common filler words so the remaining words are more meaningful. Uses a stopword list p

stopwords_en <- get_stopwords(language = "en", source = "snowball")

cat("Number of stopwords:", nrow(stopwords_en), "\n")
```

```
## Number of stopwords: 175
```

```r
head(stopwords_en, 20)
```

```
## # A tibble: 20 x 2
##    word       lexicon
##    <chr>      <chr>
##  1 i          snowball
##  2 me         snowball
##  3 my         snowball
##  4 myself     snowball
##  5 we         snowball
##  6 our        snowball
##  7 ours       snowball
##  8 ourselves  snowball
##  9 you        snowball
## 10 your       snowball
## 11 yours      snowball
## 12 yourself   snowball
## 13 yourselves snowball
## 14 he         snowball
## 15 him        snowball
## 16 his        snowball
## 17 himself    snowball
## 18 she        snowball
## 19 her        snowball
## 20 hers       snowball
```

```r
job_words_clean <- job_words |>
  anti_join(stopwords_en, by = "word") |>
  filter(str_length(word) > 2) |>
  filter(!str_detect(word, "^[0-9]+$")) |>
  # Also remove a few text artifacts we found during review that are not real skills or useful words

  filter(!word %in% c("eg", "pgi", "ive", "jee", "mis", "san"))
 # Examples: 'eg' came from e.g.,

cat("Tokens before stopword removal:", nrow(job_words), "\n")
```

```
## Tokens before stopword removal: 92499
```

```r
cat("Tokens after stopword removal: ", nrow(job_words_clean), "\n")
```

```
## Tokens after stopword removal:  64767
```

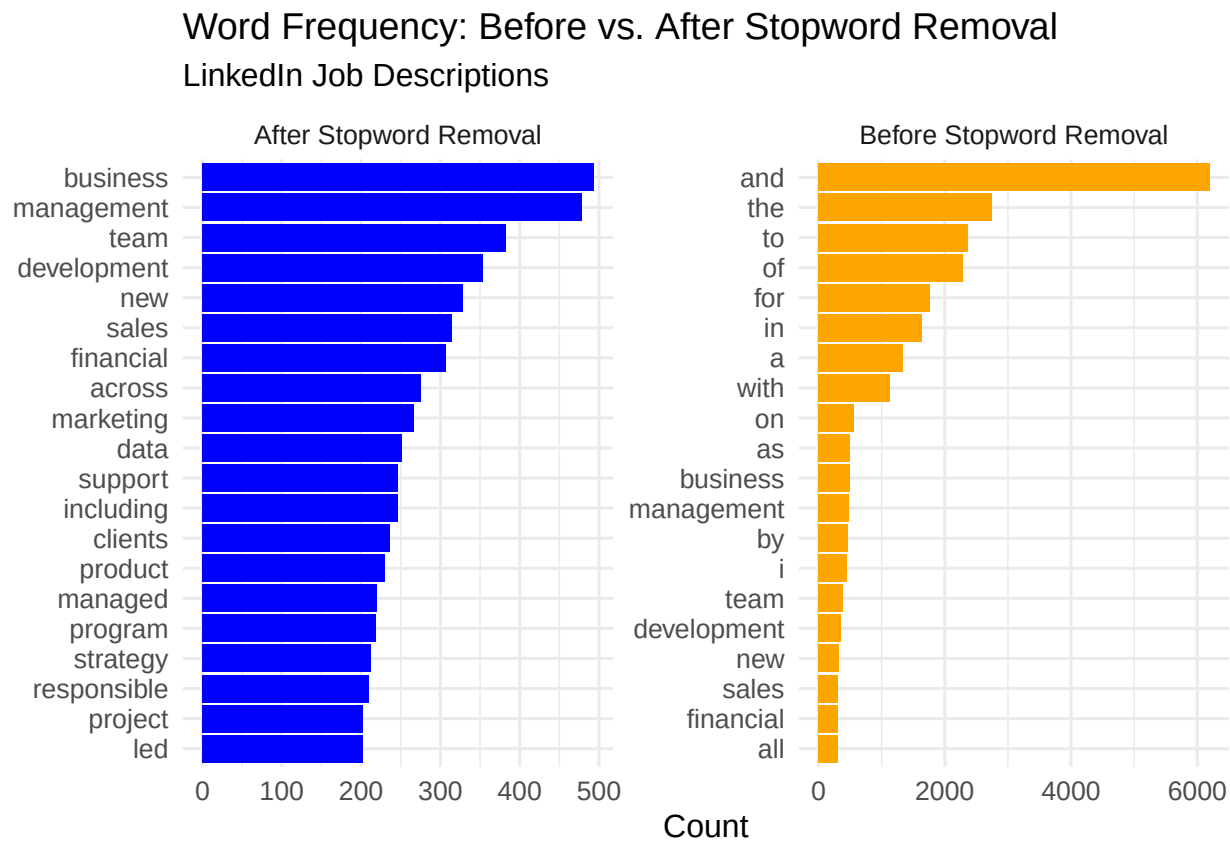
```r
# Output meaning: this number dropped because we removed filler words and kept more useful terms.
```

```r
# Look at the most common meaningful words after stopword removal
job_words_clean |>
  count(word, sort = TRUE) |>
  head(20)
```

```
##           word   n
## 1     business 493
## 2   management 478
## 3         team 383
## 4  development 354
## 5          new 328
## 6        sales 314
## 7    financial 307
## 8       across 275
## 9    marketing 267
## 10        data 252
## 11     support 247
## 12   including 246
## 13     clients 237
## 14     product 230
## 15     managed 220
## 16     program 219
## 17    strategy 213
## 18 responsible 210
## 19         led 202
## 20     project 202
```

```r
# Output meaning: after cleaning, the top words are mostly business and management terms.
# That matches what we expected from the LinkedIn profiles in this dataset.
```

```r
# Compare top word frequencies before and after stopword removal to show that cleaning improved the tex
bind_rows(
  job_words |> count(word, sort = TRUE) |> slice(1:20) |> mutate(stage = "Before Stopword Removal"),
  job_words_clean |> count(word, sort = TRUE) |> slice(1:20) |> mutate(stage = "After Stopword Removal")
  mutate(word = reorder_within(word, n, stage)) |>
  ggplot(aes(x = word, y = n, fill = stage)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~stage, scales = "free") +
  scale_x_reordered() +
  coord_flip() +
  scale_fill_manual(values = c("Before Stopword Removal" = "orange",
                               "After Stopword Removal"  = "blue")) +
  labs(
    title = "Word Frequency: Before vs. After Stopword Removal",
    subtitle = "LinkedIn Job Descriptions",
    x = NULL, y = "Count") +
  theme_minimal(base_size = 12)
```




```r
# Step 4: use stemming as a first pass to group similar word forms together
# Checking what it looks like before
cat("Unique words before stemming:", n_distinct(job_words_clean$word), "\n")
```

```
## Unique words before stemming: 9601
```

```r
job_words_stemmed <- job_words_clean |>
  mutate(word_stem = wordStem(word, language = "english"))

cat("Unique stems after stemming: ", n_distinct(job_words_stemmed$word_stem), "\n")
```

```
## Unique stems after stemming:  6574
```

```r
job_words_stemmed |>
  filter(word != word_stem) |>
  distinct(word, word_stem) |>
  head(20)
```

```
##            word word_stem
## 1       manager     manag
## 2          oaks       oak
## 3     principal   princip
## 4     investors  investor
## 5   diversified diversifi
```

```
## 6    management     manag
## 7     financial   financi
## 8    investment    invest
## 9  capabilities    capabl
## 10        range      rang
## 11       equity    equiti
## 12        fixed       fix
## 13       income     incom
## 14       estate     estat
## 15  investments    invest
## 16  specialized   special
## 17    expertise  expertis
## 18     currency   currenc
## 19   allocation     alloc
## 20       stable     stabl
```

```
# Output meaning: fewer unique stems means similar words are being grouped together.
```

```
# Compare vocabulary size before and after stemming to see how much the word list got reduced
cat("Vocabulary reduction:",
    round((1 - n_distinct(job_words_stemmed$word_stem)/n_distinct(job_words_clean$word)) * 100, 1),
    "%\n")
```

```
## Vocabulary reduction: 31.5 %
```

```
# Review a sample of stems to make sure the stemming step still looks reasonable
job_words_stemmed |>
  distinct(word, word_stem) |>
  filter(str_length(word_stem) <= 3) |>
  head(15)
```

```
##       word word_stem
## 1     oaks       oak
## 2      big       big
## 3    fixed       fix
## 4      led       led
## 5      web       web
## 6      two       two
## 7      may       may
## 8      ibm       ibm
## 9  running       run
## 10     gas       gas
## 11     key       key
## 12 bidding       bid
## 13     epc       epc
## 14     etc       etc
## 15     due       due
```

```
# Output meaning: stemming gives us a smaller and cleaner vocabulary, which helps later skill matching
```

```r
# Step 5: create bigrams so we can keep common two-word phrases, not just single words
library(tidyr)
job_bigrams <- job_text |>
  unnest_tokens(output = bigram, input = text_clean, token = "ngrams", n = 2) |>
  filter(!is.na(bigram))

# Quick check so we can see how many total tokens we created and what they look likes
job_bigrams |>
  count(bigram, sort = TRUE) |>
  head(20)
```

```
##             bigram   n
## 1           of the 316
## 2           in the 298
## 3          for the 252
## 4  responsible for 203
## 5           to the 120
## 6             as a 108
## 7         with the 104
## 8          and the  92
## 9        to ensure  84
## 10           for a  75
## 11             of a  73
## 12     real estate  73
## 13          with a  72
## 14             in a  70
## 15          on the  65
## 16             is a  64
## 17  management and  64
## 18         such as  63
## 19    social media  61
## 20 development and  59
```

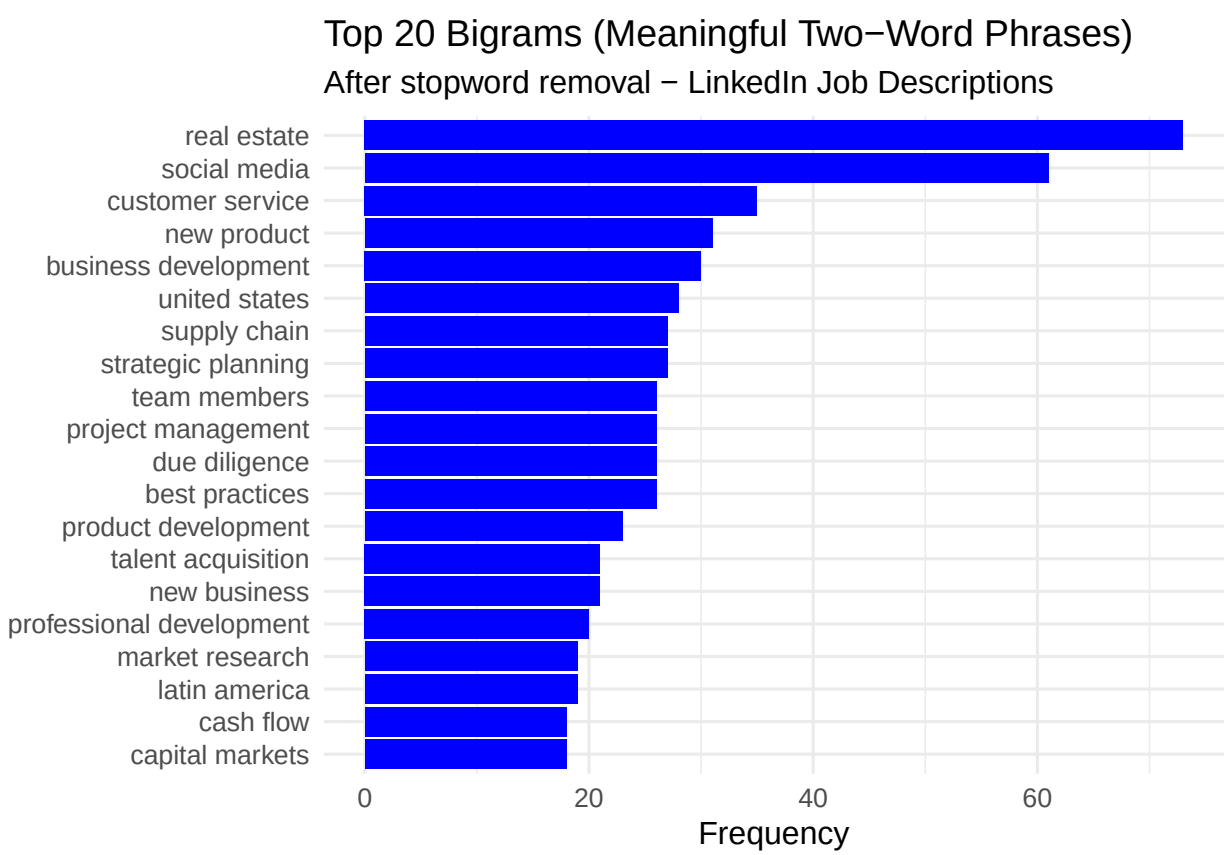
```r
# Remove bigrams that include stopwords because those phrases are less useful
job_bigrams_clean <- job_bigrams |>
  separate(bigram, into = c("word1", "word2"), sep = " ") |>
  anti_join(stopwords_en, by = c("word1" = "word")) |>
  anti_join(stopwords_en, by = c("word2" = "word")) |>
  unite(bigram, word1, word2, sep = " ")

# Quick check so we can see how many total tokens we created and what they look likes
job_bigrams_clean |>
  count(bigram, sort = TRUE) |>
  head(20)
```

```
##                      bigram  n
## 1               real estate 73
## 2              social media 61
## 3          customer service 35
## 4               new product 31
## 5      business development 30
## 6             united states 28
## 7        strategic planning 27
```

```
## 8              supply chain 27
## 9            best practices 26
## 10            due diligence 26
## 11       project management 26
## 12             team members 26
## 13      product development 23
## 14             new business 21
## 15       talent acquisition 21
## 16 professional development 20
## 17            latin america 19
## 18          market research 19
## 19          capital markets 18
## 20                cash flow 18
```

```
job_bigrams_clean |>
  count(bigram, sort = TRUE) |>
  head(20) |>
  mutate(bigram = fct_reorder(bigram, n)) |>
  ggplot(aes(x = bigram, y = n)) +
  geom_col(fill = "blue") +
  coord_flip() +
  labs(
    title = "Top 20 Bigrams (Meaningful Two-Word Phrases)",
    subtitle = "After stopword removal - LinkedIn Job Descriptions",
    x = NULL, y = "Frequency") +
  theme_minimal(base_size = 12)
```



```r
# Step 6: build the final processed text table that combines all of the main cleaning steps
# This becomes our final token-level dataset for feature creation
processed_text <- job_text |>
  unnest_tokens(output = word, input = text_clean, token = "words") |>
  anti_join(get_stopwords(source = "snowball"), by = "word") |>
  filter(str_length(word) > 2) |>
  filter(!str_detect(word, "^[0-9]+$")) |>
  mutate(word_stem = wordStem(word, language = "english")) |>
  count(linkedinUrl, word_stem, sort = TRUE) |>
  rename(word = word_stem)

cat("Final processed text dimensions:", nrow(processed_text), "rows\n")
```

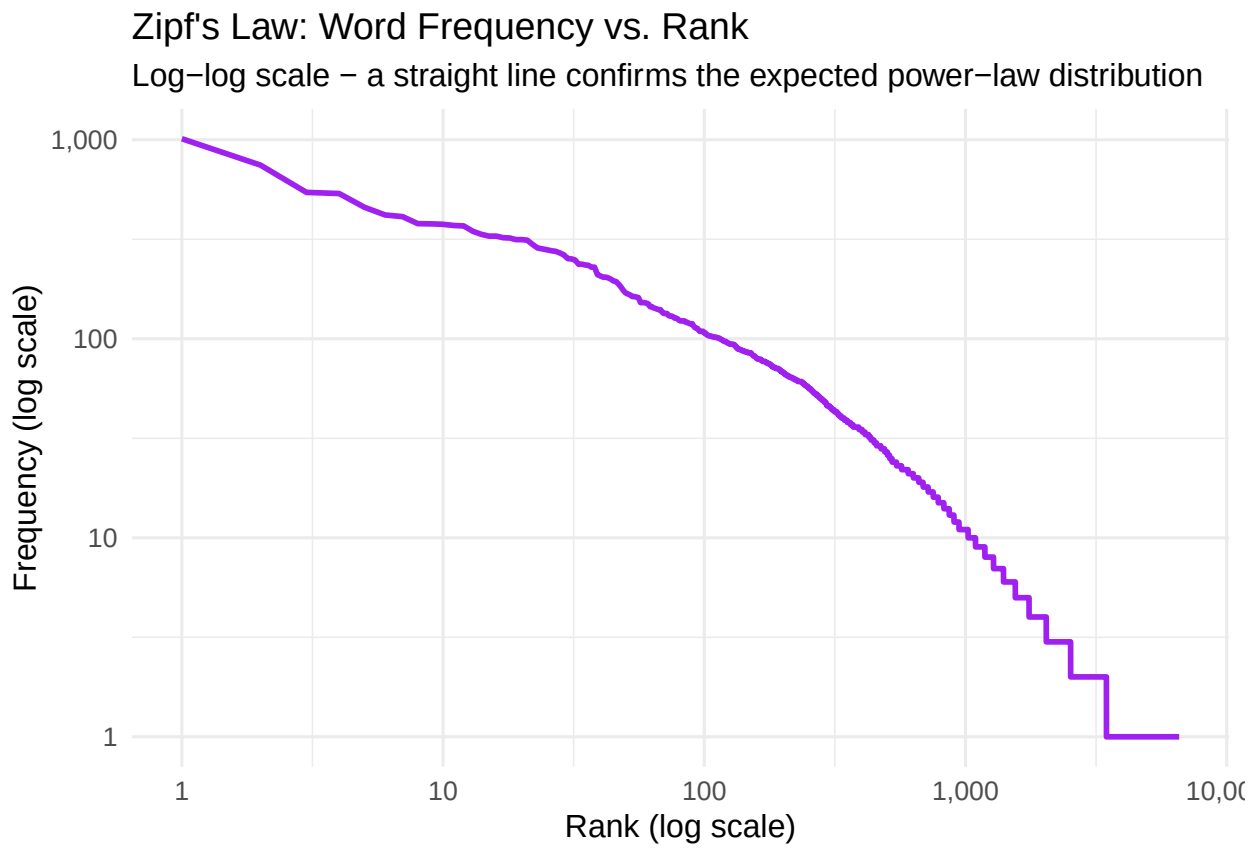
```
## Final processed text dimensions: 38515 rows
```

```r
glimpse(processed_text)
```

```
## Rows: 38,515
## Columns: 3
## $ linkedinUrl <chr> "https://www.linkedin.com/in/stefanie-smedstad-phr-shrm-cp~
## $ word        <chr> "talent", "manag", "student", "project", "tax", "manag", "~
## $ n           <int> 62, 53, 42, 39, 38, 37, 34, 30, 30, 29, 29, 28, 27, 25, 25~
```

```r
# Output meaning: After all the cleaning steps, we now have a token-level dataset with 38,515 rows.
# Each row represents a word (after cleaning and stemming) tied to a specific LinkedIn profile.
# The 'n' column shows how many times that word appears in that profile's job description.
# This tells us the text has been successfully cleaned, reduced to meaningful words, and is now
# structured in a way we can use for feature creation
```

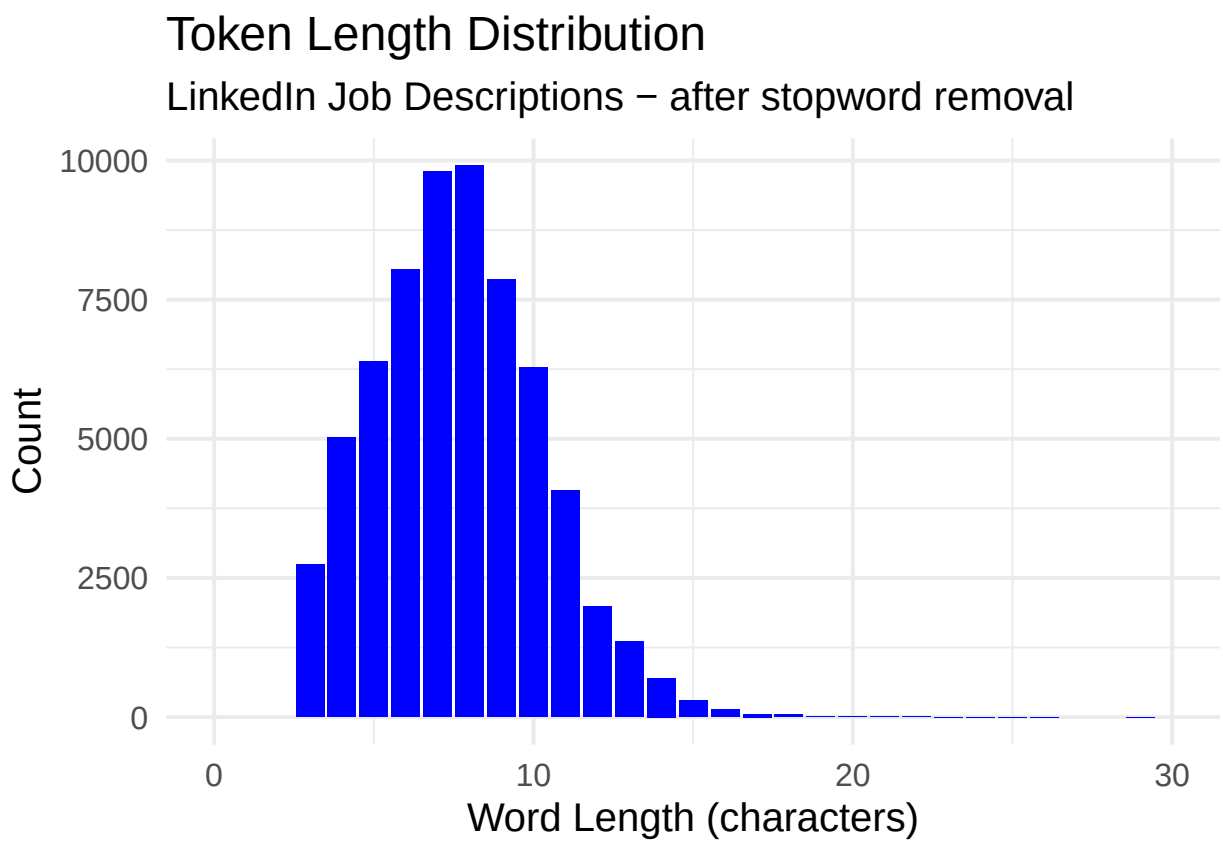
```r
# Final quality checks so we can show the processed text looks reasonable before modeling
# Check 1: Zipf's Law plot to see whether word frequency follows the pattern we usually expect in natur
processed_text |>
  count(word, wt = n, sort = TRUE) |>
  mutate(rank = row_number()) |>
  ggplot(aes(x = rank, y = n)) +
  geom_line(color = "purple", linewidth = 1) +
  scale_x_log10(labels = label_comma()) +
  scale_y_log10(labels = label_comma()) +
  labs(
    title = "Zipf's Law: Word Frequency vs. Rank",
    subtitle = "Log-log scale \u2014 a straight line confirms the expected power-law distribution",
    x = "Rank (log scale)", y = "Frequency (log scale)") +
  theme_minimal(base_size = 12)
```




```
# It's mostly a straight line on the log-log scale, so it follows Zipf's Law. That tells me the text pr
```

```
# Check 2: look at word lengths after cleaning to make sure the token sizes seem normal
job_words_clean |>
  mutate(word_length = str_length(word)) |>
  count(word_length) |>
  ggplot(aes(x = word_length, y = n)) +
  geom_col(fill = "blue", width = 0.9) +
  xlim(0,30) +
  labs(
    title = "Token Length Distribution",
    subtitle = "LinkedIn Job Descriptions - after stopword removal",
    x = "Word Length (characters)", y = "Count") +
  theme_minimal(base_size = 15)
```

```
## Warning: Removed 5 rows containing missing values or values outside the scale range
## (`geom_col()`).
```




```r
# Check 3: manually review a random sample of processed words for anything weird
set.seed(621)
processed_text |>
  distinct(word) |>
  slice_sample(n = 20) |>
  pull(word) |>
  sort()
```

```
##  [1] "basebal"           "call"              "cash"
##  [4] "cloudfront"        "direct"            "dispatch"
##  [7] "flyersmail"        "identif"           "infrastructur"
## [10] "kepwar"            "mainten"           "mealsbeveragelodg"
## [13] "mix"               "oral"              "process"
## [16] "repli"             "roadmapsdiscov"    "trainer"
## [19] "uw"                "workflow"
```

```r
# Output meaning: most stems look like real professional terms.
# We did find a few merged words like mealsbeveragelodg, roadmapsdiscov, and flyersmail.
# Those are most likely formatting issues from scraped LinkedIn text
# planning to clean it up more in CP8
```

Part 4: Feature Engineering and Variable Creation

```r
# Step 1: look at the top 100 self-reported skills so we can build a skill dictionary from common terms
skills_clean |>
  count(skill, sort = TRUE) |>
  head(100) |>
  pull(skill)
```

```
##   [1] "Leadership"                       "Microsoft Excel"
##   [3] "Project Management"               "Microsoft Office"
##   [5] "Management"                       "Strategic Planning"
##   [7] "Customer Service"                 "Communication"
##   [9] "Public Speaking"                  "Financial Analysis"
##  [11] "Marketing"                        "Sales"
##  [13] "Data Analysis"                    "Finance"
##  [15] "Teamwork"                         "Microsoft Word"
##  [17] "Program Management"               "Social Media"
##  [19] "Accounting"                       "PowerPoint"
##  [21] "Research"                         "Strategy"
##  [23] "Business Strategy"                "Marketing Strategy"
##  [25] "Business Development"             "Microsoft PowerPoint"
##  [27] "Problem Solving"                  "Team Leadership"
##  [29] "Cross-functional Team Leadership" "Financial Modeling"
##  [31] "Operations Management"            "Team Building"
##  [33] "Time Management"                  "Event Planning"
##  [35] "Budgets"                          "Entrepreneurship"
##  [37] "Social Media Marketing"           "Investments"
##  [39] "Market Research"                  "Process Improvement"
##  [41] "Recruiting"                       "Training"
##  [43] "Analytical Skills"                "Project Planning"
##  [45] "SQL"                              "Financial Accounting"
##  [47] "Financial Reporting"              "New Business Development"
##  [49] "Analysis"                         "Coaching"
##  [51] "Tableau"                          "Change Management"
##  [53] "Organization Skills"              "Sales Management"
##  [55] "Advertising"                      "Negotiation"
##  [57] "Product Management"               "Risk Management"
##  [59] "Business Process Improvement"     "Contract Negotiation"
##  [61] "Corporate Finance"                "Real Estate"
##  [63] "Auditing"                         "Due Diligence"
##  [65] "Fundraising"                      "Interpersonal Skills"
##  [67] "Management Consulting"            "Python (Programming Language)"
##  [69] "Start-ups"                        "Asset Management"
##  [71] "Community Outreach"               "Product Development"
##  [73] "Valuation"                        "Banking"
##  [75] "Business Analysis"                "Event Management"
##  [77] "Higher Education"                 "Leadership Development"
##  [79] "Portfolio Management"             "Artificial Intelligence (AI)"
##  [81] "Digital Marketing"                "Healthcare"
##  [83] "Manufacturing"                    "Social Networking"
##  [85] "Supply Chain Management"          "Teaching"
##  [87] "Writing"                          "Account Management"
##  [89] "Bloomberg"                        "Business Planning"
##  [91] "Commercial Real Estate"           "Competitive Analysis"
##  [93] "Critical Thinking"                "Customer Satisfaction"
```

```
##  [95] "Equities"                          "Government"
##  [97] "Internal Controls"                 "Java"
##  [99] "Presentations"                     "Public Relations"
```

```r
skills_dict <- skills_clean |>
  count(skill, sort = TRUE) |>
  head(100) |>
  mutate(
    skill_raw = tolower(skill),
    skill_category = case_when(
      # Technical & Data
      skill %in% c("Data Analysis", "Analytics", "SQL", "Python",
                   "Machine Learning", "Data Science", "Tableau",
                   "Analysis", "Analytical Skills", "Research",
                   "Python (Programming Language)", "Java",
                   "Artificial Intelligence (AI)", "Bloomberg",
                   "Financial Modeling", "Competitive Analysis",
                   "Market Research") ~ "Technical & Data",
      # Tools & Software
      skill %in% c("Microsoft Excel", "Microsoft Office", "Microsoft Word",
                   "PowerPoint", "Microsoft PowerPoint", "Social Media",
                   "Salesforce", "Social Media Marketing") ~ "Tools",
      # Business & Management
      skill %in% c("Project Management", "Strategic Planning", "Management",
                   "Program Management", "Business Development",
                   "Operations Management", "Strategy", "Business Strategy",
                   "Change Management", "Risk Management", "Product Management",
                   "Process Improvement", "Business Process Improvement",
                   "Project Planning", "Asset Management", "Account Management",
                   "Event Management", "Portfolio Management",
                   "Supply Chain Management", "Contract Negotiation",
                   "Negotiation", "Business Planning",
                   "New Business Development", "Management Consulting",
                   "Leadership Development", "Cross-functional Team Leadership",
                   "Sales Management", "Time Management",
                   "Organization Skills") ~ "Business & Management",
      # Soft Skills
      skill %in% c("Leadership", "Communication", "Public Speaking",
                   "Teamwork", "Customer Service", "Team Leadership",
                   "Coaching", "Problem Solving", "Team Building",
                   "Interpersonal Skills", "Critical thinking",
                   "Training", "Teaching", "Presentations",
                   "Customer Satisfaction", "Social Networking",
                   "Community Outreach", "Public Relations") ~ "Soft Skills",
      # Business Functions
      skill %in% c("Marketing", "Sales", "Finance", "Accounting",
                   "Financial Analysis", "Financial Reporting",
                   "Financial Accounting", "Corporate Finance",
                   "Auditing", "Investments", "Valuation",
                   "Banking", "Equities", "Real Estate",
                   "Commercial Real Estate", "Due Diligence",
                   "Budgets", "Internal Controls",
                   "Marketing Strategy", "Advertising",
                   "Digital Marketing", "Fundraising",
```

```r
                    "Business Analysis", "Product Development",
                    "Entrepreneurship", "Start-ups",
                    "Event Planning", "Recruiting",
                    "Higher Education", "Healthcare",
                    "Manufacturing", "Government",
                    "Writing") ~ "Business Functions",
      TRUE ~ "Other"
    )
  ) |>
  select(skill_raw, skill_category)

cat("Skills in dictionary:", nrow(skills_dict), "\n")
```

```
## Skills in dictionary: 100
```

```r
skills_dict |> count(skill_category)
```

```
##          skill_category  n
## 1 Business & Management 29
## 2    Business Functions 33
## 3                 Other  1
## 4           Soft Skills 17
## 5      Technical & Data 13
## 6                 Tools  7
```

```r
# Output meaning: this shows how many top skills made it into our dictionary for matching.
```

```r
# Step 2: create a function that checks whether a job description contains any skills from our dictiona
# This function scans each description and returns the skills that show up in the text

extract_skills_keyword <- function(text, dictionary) {
  text_lower <- str_to_lower(text)

  dictionary |>
    filter(map_lgl(skill_raw, function(skill) {
      pattern <- if (str_length(skill) <= 2) {
        str_glue("\\b{skill}\\b")
      } else {
        skill
      }
      str_detect(text_lower, regex(pattern, ignore_case = TRUE))
    }))
}

# Test the function on a few profiles first so we can make sure the extraction logic is working
for(i in 1:10) {
  desc <- exp_clean$jobDescription[i]
  if(!is.na(desc) & desc != "") {
    skills_found <- extract_skills_keyword(desc, skills_dict)
    if(nrow(skills_found) > 0) {
      cat("Profile", i, "- Found", nrow(skills_found), "skills:\n")
      print(skills_found |> select(skill_raw, skill_category))
```

```r
      cat("\n")
    }
  }
}
```

```
## Profile 2 - Found 12 skills:
##             skill_raw        skill_category
## 1          leadership           Soft Skills
## 2    microsoft office                 Tools
## 3          management Business & Management
## 4  strategic planning Business & Management
## 5       communication           Soft Skills
## 6            research      Technical & Data
## 7            strategy Business & Management
## 8         investments    Business Functions
## 9         real estate    Business Functions
## 10   asset management Business & Management
## 11          bloomberg      Technical & Data
## 12               java      Technical & Data
##
## Profile 3 - Found 2 skills:
##          skill_raw        skill_category
## 1       management Business & Management
## 2 asset management Business & Management
##
## Profile 4 - Found 3 skills:
##          skill_raw        skill_category
## 1       management Business & Management
## 2    due diligence    Business Functions
## 3 asset management Business & Management
##
## Profile 5 - Found 1 skills:
##    skill_raw     skill_category
## 1 healthcare Business Functions
##
## Profile 7 - Found 5 skills:
##              skill_raw        skill_category
## 1           management Business & Management
## 2            marketing    Business Functions
## 3                sales    Business Functions
## 4     asset management Business & Management
## 5 portfolio management Business & Management
##
## Profile 8 - Found 9 skills:
##               skill_raw        skill_category
## 1            leadership           Soft Skills
## 2         communication           Soft Skills
## 3             marketing    Business Functions
## 4                 sales    Business Functions
## 5              research      Technical & Data
## 6              strategy Business & Management
## 7              training           Soft Skills
## 8           real estate    Business Functions
```

```
## 9 customer satisfaction           Soft Skills
##
## Profile 10 - Found 4 skills:
##          skill_raw        skill_category
## 1       management Business & Management
## 2    communication           Soft Skills
## 3 asset management Business & Management
## 4       government    Business Functions
```

```r
# Step 3: apply the skill extraction function to every profile with a job description
library(purrr)

extracted_skills <- exp_clean |>
  filter(!is.na(jobDescription) & jobDescription != "") |>
  select(linkedinUrl, jobDescription) |>
  mutate(
    skills = map(jobDescription, extract_skills_keyword,
                 dictionary = skills_dict) ) |>
  select(-jobDescription) |>
  unnest(skills)

cat("Total skill extractions:", nrow(extracted_skills), "\n")
```

```
## Total skill extractions: 1670
```

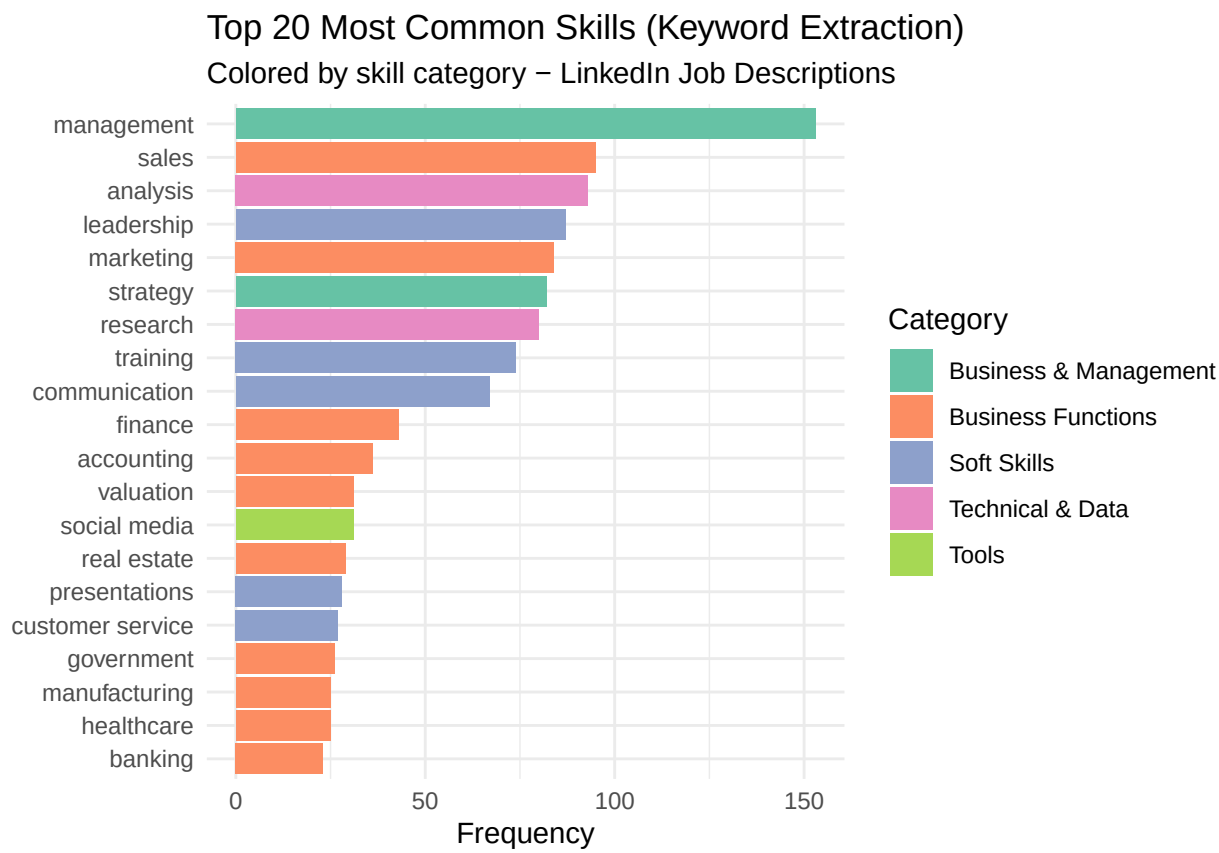
```r
# Output meaning: this is the total number of skill matches found across all job descriptions.
cat("Profiles with at least one skill extracted:",
    n_distinct(extracted_skills$linkedinUrl), "\n")
```

```
## Profiles with at least one skill extracted: 267
```

```r
# Output meaning:
# Keyword matching extracted 1,670 skills across 267 profiles
# skills were found in 267 out of 316 profiles with descriptions
```

```r
# Visualize the extraction results so we can see which skills and categories appear most often
library(ggplot2)
library(forcats)

extracted_skills |>
  count(skill_raw, skill_category, sort = TRUE) |>
  head(20) |>
  mutate(skill_raw = fct_reorder(skill_raw, n)) |>
  ggplot(aes(x = skill_raw, y = n, fill = skill_category)) +
  geom_col() +
  coord_flip() +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title    = "Top 20 Most Common Skills (Keyword Extraction)",
    subtitle = "Colored by skill category - LinkedIn Job Descriptions",
    x = NULL, y = "Frequency", fill = "Category") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "right")
```




```r
# Step 4: build TF-IDF features so we can measure which words matter most within each profile
# This follows the professor's Module 3 process for turning cleaned text into model-ready features
word_counts <- processed_text |>
  count(linkedinUrl, word, sort = TRUE)

cat("Unique words (vocabulary size):", n_distinct(word_counts$word), "\n")
```

```
## Unique words (vocabulary size): 6579
```

```r
# Output meaning: vocabulary size tells us how many distinct word features we have after preprocessing.
cat("Document-word pairs:           ", nrow(word_counts), "\n")
```

```
## Document-word pairs:            38515
```

```r
cat("Sparsity:                      ",
    round((1 - nrow(word_counts) /
           (n_distinct(word_counts$linkedinUrl) *
            n_distinct(word_counts$word))) * 100, 1), "%\n")
```

```
## Sparsity:                       98.1 %
```

```r
tfidf <- word_counts |>
  bind_tf_idf(term = word, document = linkedinUrl, n = n)
```

```r
tfidf |>
  arrange(desc(tf_idf)) |>
  head(15)
```

```
##                                                  linkedinUrl
## 1        https://www.linkedin.com/in/tobias-cremer-80252610b/
## 2        https://www.linkedin.com/in/tobias-cremer-80252610b/
## 3        https://www.linkedin.com/in/tobias-cremer-80252610b/
## 4     https://www.linkedin.com/in/ananya-madireddy-439891244/
## 5                       https://www.linkedin.com/in/sameehah/
## 6          https://www.linkedin.com/in/benjamin-ma-39a1aa219/
## 7     https://www.linkedin.com/in/nicholas-martini-b960501b6/
## 8  https://www.linkedin.com/in/richard-descano-cpa-b36034177/
## 9               https://www.linkedin.com/in/brooke-wood-0015/
## 10              https://www.linkedin.com/in/brooke-wood-0015/
## 11              https://www.linkedin.com/in/brooke-wood-0015/
## 12  https://www.linkedin.com/in/david-kingston-cpa-1501b7114/
## 13        https://www.linkedin.com/in/nissim-attali-a9b5a011/
## 14    https://www.linkedin.com/in/ananya-madireddy-439891244/
## 15                  https://www.linkedin.com/in/angela-s-ryu/
##                      word n        tf      idf    tf_idf
## 1           auenausschuss 1 0.3333333 5.752573 1.9175242
## 2                    mitg 1 0.3333333 5.752573 1.9175242
## 3  verteidigungsausschuss 1 0.3333333 5.752573 1.9175242
## 4              internship 1 0.5000000 2.979984 1.4899920
## 5                  ciocca 1 0.2500000 5.752573 1.4381432
## 6                     fig 1 0.2500000 5.059425 1.2648564
## 7                  logist 1 0.5000000 2.494476 1.2472381
## 8                  merger 1 0.3333333 3.673131 1.2243770
## 9           congresswoman 1 0.2000000 5.752573 1.1505145
## 10                 elissa 1 0.2000000 5.752573 1.1505145
## 11                slotkin 1 0.2000000 5.752573 1.1505145
## 12      accountingfinanci 1 0.2000000 5.752573 1.1505145
## 13                  numer 1 0.3333333 3.267666 1.0892220
## 14                  track 1 0.5000000 2.089011 1.0445055
## 15                marshal 1 0.1666667 5.752573 0.9587621
```

```r
# Output meaning: We have 6,579 unique words after cleaning, which is our total vocabulary size for the
# There are 38,515 document-word pairs, meaning these words appear across profiles that many times.
# The sparsity is 98.1%, which is expected for text data. Most words do not appear in most profiles.
# This tells us our dataset is large, detailed, and ready for TF-IDF, where we can identify which
# words are most important for each profile.
```

```r
# Convert TF-IDF results into a wide feature matrix where each row is a profile and each column is a wo
# This matches the professor's Module 3 feature matrix step
feature_matrix <- tfidf |>
  select(linkedinUrl, word, tf_idf) |>
  pivot_wider(
    names_from  = word,
    values_from = tf_idf,
    values_fill = 0)
```

```r
cat("Feature matrix dimensions:", nrow(feature_matrix),
    "rows x", ncol(feature_matrix), "columns\n")
```

```
## Feature matrix dimensions: 315 rows x 6580 columns
```

```r
# Output meaning: each row is one LinkedIn profile and each column is a TF-IDF word feature.
```

```r
# Check the TF-IDF value range so we can confirm the feature values look normal
summary(tfidf$tf_idf)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.0007462 0.0079016 0.0131294 0.0231490 0.0232084 1.9175242
```

```r
cat("Min TF-IDF:", min(tfidf$tf_idf), "\n")
```

```
## Min TF-IDF: 0.0007462095
```

```r
cat("Max TF-IDF:", max(tfidf$tf_idf), "\n")
```

```
## Max TF-IDF: 1.917524
```

```r
cat("Mean TF-IDF:", round(mean(tfidf$tf_idf), 4), "\n")
```

```
## Mean TF-IDF: 0.0231
```

```r
# Output meaning:
# A max TF-IDF a little above 1 is normal when using bind_tf_idf() based off guide
# So we do not need to rescale anything here because the values are still consistent across profiles.
```

Part 5: Labeling and Ground Truth

```r
# This section follows the professor's Module 4 labeling and evaluation steps
# Ground truth idea: the self-reported skills list is our answer key
# We are using weak supervision, which means labels are generated automatically from dictionary matches

# Step 1: set up the ground truth table we will compare against
# The skills file is treated like our best available answer key
# These are skills users already listed on their own LinkedIn profiles
# We compare our extracted skills to those self-reported skills to see how well the extraction worked

ground_truth <- skills_clean |>
  mutate(skill_true = tolower(skill)) |>
  select(linkedinUrl, skill_true)

cat("Total ground truth skill entries:", nrow(ground_truth), "\n")
```

```
## Total ground truth skill entries: 6364
```

```r
cat("Profiles with ground truth skills:", n_distinct(ground_truth$linkedinUrl), "\n")
```

```
## Profiles with ground truth skills: 402
```

```r
# Output meaning: this is the number of self-reported skill records we are treating as the answer key.
```

```r
# Step 2: create BIO labels for token-level skill tagging
# This follows the professor's token labeling example from Module 4
# BIO labels tell us whether a token starts a skill, continues a skill, or is outside a skill
# B = the beginning of a skill phrase
# I = the next word(s) inside that same skill phrase
# O = not part of any skill phrase

create_bio_labels <- function(text, dictionary) {
  tokens <- tibble(
    token    = unlist(str_split(str_to_lower(text), "\\s+")),
    position = seq_along(token))
  tokens <- tokens |> mutate(bio_label = "O")

  for (skill in dictionary$skill_raw) {
    skill_words <- unlist(str_split(skill, "\\s+"))
    n_words     <- length(skill_words)

    for (start_pos in seq_len(nrow(tokens) - n_words + 1)) {
      window <- tokens$token[start_pos:(start_pos + n_words - 1)]
      if (all(window == skill_words)) {
        tokens$bio_label[start_pos] <-
          paste0("B-", str_to_upper(str_replace_all(skill, "\\s+", "_")))
        if (n_words > 1) {
          tokens$bio_label[(start_pos + 1):(start_pos + n_words - 1)] <-
            paste0("I-", str_to_upper(str_replace_all(skill, "\\s+", "_")))
        }
      }
    }
  }
  tokens
}

# Show one example profile so we can see how BIO labels look in practice
bio_example <- create_bio_labels(
  exp_clean$jobDescription[2],
  skills_dict)

# Only display the tokens that got a skill label, not the full text
bio_example |>
  filter(bio_label != "O") |>
  head(20)
```

```
## # A tibble: 13 x 3
##    token         position bio_label
##    <chr>            <int> <chr>
##  1 asset                8 B-ASSET_MANAGEMENT
```

```
##  2 management           9 I-ASSET_MANAGEMENT
##  3 real                30 B-REAL_ESTATE
##  4 estate              31 I-REAL_ESTATE
##  5 management          45 B-MANAGEMENT
##  6 management          56 B-MANAGEMENT
##  7 leadership         133 B-LEADERSHIP
##  8 java               158 B-JAVA
##  9 management         308 B-MANAGEMENT
## 10 strategy           314 B-STRATEGY
## 11 communication      604 B-COMMUNICATION
## 12 strategic          618 B-STRATEGIC_PLANNING
## 13 planning           619 I-STRATEGIC_PLANNING
```

```r
# Check the label mix across a small sample of profiles
bio_labeled <- exp_clean |>
  filter(!is.na(jobDescription) & jobDescription != "") |>
  head(5) |>
  mutate(bio_tokens = map(jobDescription, create_bio_labels, dictionary = skills_dict)) |>
  select(linkedinUrl, bio_tokens) |>
  unnest(bio_tokens)

bio_labeled |>
  mutate(label_type = case_when(
    str_starts(bio_label, "B-") ~ "Begin (B)",
    str_starts(bio_label, "I-") ~ "Inside (I)",
    TRUE ~ "Outside (O)"
  )) |>
  count(label_type) |>
  mutate(pct = scales::percent(n / sum(n)))
```

```
## # A tibble: 3 x 3
##   label_type      n pct
##   <chr>       <int> <chr>
## 1 Begin (B)      13 1.36%
## 2 Inside (I)      5 0.52%
## 3 Outside (O)   936 98.11%
```

```r
# Output meaning: Most of the tokens are labeled as Outside (O) (~98%), which means they are not
# part of any skill phrase.
# Only a small portion are Begin (B) or Inside (I), so actual skill terms are pretty rare in
# the text.
# This kind of imbalance is expected for this type of task, since most words in a job
# description are not skills.
```

```r
# Step 3: evaluate how well our extracted skills match the ground truth skills
# This follows the evaluation setup from the professor's Module 4 guide
# We compare extracted skills to self-reported skills profile by profile

# First keep only profiles that appear in both datasets so the comparison is fair
common_profiles <- intersect(
  unique(extracted_skills$linkedinUrl),
  unique(ground_truth$linkedinUrl))
```

```r
cat("Profiles in both extracted and ground truth:",
    length(common_profiles), "\n")
```

```
## Profiles in both extracted and ground truth: 253
```

```r
# True Positives = skills we extracted that also appear in the self-reported skill list
tp_df <- inner_join(
  extracted_skills |> filter(linkedinUrl %in% common_profiles),
  ground_truth     |> filter(linkedinUrl %in% common_profiles),
  by = c("linkedinUrl", "skill_raw" = "skill_true"))

# False Positives = skills we extracted that do not appear in the self-reported skill list
fp_df <- anti_join(
  extracted_skills |> filter(linkedinUrl %in% common_profiles),
  ground_truth     |> filter(linkedinUrl %in% common_profiles),
  by = c("linkedinUrl", "skill_raw" = "skill_true"))

# False Negatives = skills listed in the ground truth that our extraction missed
fn_df <- anti_join(
  ground_truth     |> filter(linkedinUrl %in% common_profiles),
  extracted_skills |> filter(linkedinUrl %in% common_profiles),
  by = c("linkedinUrl", "skill_true" = "skill_raw"))

TP <- nrow(tp_df)
FP <- nrow(fp_df)
FN <- nrow(fn_df)

precision <- round(TP / (TP + FP), 3)
recall    <- round(TP / (TP + FN), 3)
f1        <- round(2 * precision * recall / (precision + recall), 3)

tibble(
  Metric = c("True Positives", "False Positives", "False Negatives",
             "Precision", "Recall", "F1 Score"),
  Value  = c(TP, FP, FN, precision, recall, f1))
```

```
## # A tibble: 6 x 2
##   Metric             Value
##   <chr>              <dbl>
## 1 True Positives   359
## 2 False Positives 1273
## 3 False Negatives 3930
## 4 Precision          0.22
## 5 Recall             0.084
## 6 F1 Score           0.122
```

```r
# Output meaning: precision tells us how many extracted skills were correct,
# recall tells us how many true skills we successfully found,
# and F1 gives one combined score balancing both.
# Our F1 score is low at 0.122, which means the model did not accurately identify skills.
```

```r
# Show a sample of missed skills so we can see what kinds of terms were not captured
cat("Skills missed (False Negatives sample):\n")
```

```
## Skills missed (False Negatives sample):
```

```r
fn_df |> head(20)
```

```
##                                          linkedinUrl              skill_true
## 1               https://www.linkedin.com/in/daleward/                    itil
## 2               https://www.linkedin.com/in/daleward/    software development
## 3               https://www.linkedin.com/in/daleward/                    sdlc
## 4               https://www.linkedin.com/in/daleward/               databases
## 5               https://www.linkedin.com/in/daleward/           it management
## 6               https://www.linkedin.com/in/daleward/             data center
## 7               https://www.linkedin.com/in/daleward/                    unix
## 8               https://www.linkedin.com/in/daleward/                  oracle
## 9               https://www.linkedin.com/in/daleward/              networking
## 10              https://www.linkedin.com/in/daleward/                 servers
## 11              https://www.linkedin.com/in/daleward/  information technology
## 12              https://www.linkedin.com/in/daleward/           it operations
## 13              https://www.linkedin.com/in/daleward/                   cobol
## 14              https://www.linkedin.com/in/daleward/       vendor management
## 15              https://www.linkedin.com/in/daleward/              sharepoint
## 16              https://www.linkedin.com/in/daleward/ java enterprise edition
## 17 https://www.linkedin.com/in/ting-qiu-cpa-6b7b2034/      financial analysis
## 18 https://www.linkedin.com/in/ting-qiu-cpa-6b7b2034/              quickbooks
## 19 https://www.linkedin.com/in/ting-qiu-cpa-6b7b2034/    financial accounting
## 20 https://www.linkedin.com/in/ting-qiu-cpa-6b7b2034/              accounting
```

Part 6: Data Partitioning

```r
# This section follows the professor's train/validation/test split setup from Module 3
# Step 1 and 2: split the profiles into training, validation, and test sets using a 70/15/15 split
# set.seed() makes the random split reproducible so we can get the same result again later
# This split matches the modeling plan we already laid out in CP6

set.seed(621)

n_profiles <- nrow(feature_matrix)
train_size <- floor(0.70 * n_profiles)
val_size   <- floor(0.15 * n_profiles)
test_size  <- n_profiles - train_size - val_size

shuffled_idx <- sample(n_profiles)

train_idx <- shuffled_idx[1:train_size]
val_idx   <- shuffled_idx[(train_size + 1):(train_size + val_size)]
test_idx  <- shuffled_idx[(train_size + val_size + 1):n_profiles]

train_data <- feature_matrix[train_idx, ]
```

```r
val_data   <- feature_matrix[val_idx, ]
test_data  <- feature_matrix[test_idx, ]

cat("Total profiles:", n_profiles, "\n")
```

```
## Total profiles: 315
```

```r
# Output meaning: these counts confirm the dataset was split into train, validation, and test sets as p
cat("Training samples:  ", nrow(train_data), "(70%)\n")
```

```
## Training samples:   220 (70%)
```

```r
cat("Validation samples:", nrow(val_data), "(15%)\n")
```

```
## Validation samples: 47 (15%)
```

```r
cat("Test samples:      ", nrow(test_data), "(15%)\n")
```

```
## Test samples:       48 (15%)
```

```r
# Step 3: check whether profiles with extracted skills are balanced against profiles without them
# Count how many profiles got at least one extracted skill
# This helps us see whether one class is much larger than the other

profiles_with_skills <- n_distinct(extracted_skills$linkedinUrl)
profiles_without_skills <- n_profiles - profiles_with_skills

cat("Profiles WITH skills extracted:", profiles_with_skills, "\n")
```

```
## Profiles WITH skills extracted: 267
```

```r
# Output meaning: this check shows whether most profiles fall into one class, which matters for modelin
cat("Profiles WITHOUT skills extracted:", profiles_without_skills, "\n")
```

```
## Profiles WITHOUT skills extracted: 48
```

```r
cat("Class ratio:", round(profiles_with_skills / n_profiles * 100, 1), "% have skills\n")
```

```
## Class ratio: 84.8 % have skills
```

```r
# Output meaning: The data is very imbalanced, with most tokens labeled as O and only a small
# portion being actual skill terms (B/I).
# This can cause the model to focus too much on the majority class and miss skills.
# To improve this, we plan to use stratified sampling during cross-validation and possibly apply
# class weights in CP8 so the model pays more attention
# to the smaller skill-related classes.
```